

A Multi-Domain Virtual Knowledge Graph Query System

Implementation Report — Phase 0 (Data & Infrastructure Preparation)
and Phase 1 (Domain-Switching Infrastructure)

Development & evaluation harness for the query recommender

Narges

Supervisor: Prof. Elisa Quintarelli

July 25, 2026

Abstract

This report documents the first two implementation phases of a thesis project centered on an LLM-based query recommender for natural-language querying over Virtual Knowledge Graphs (VKGs). **Phase 0** stands up two independent VKGs — a synthetic *Movie* domain and a real *Tourism* domain (Verona) — each served through an Ontop OBDA endpoint over a containerized PostgreSQL/PostGIS database. **Phase 1** adds a domain-switching mechanism on top of the existing “Indeewari” NL2SPARQL pipeline so it can point at either VKG. Both phases are complete and verified end-to-end. Following a scope decision made with the supervisor after this work was completed (Section 1.1), neither phase is a thesis deliverable in its own right: they exist solely as an internal data/infrastructure *harness* that the actual contribution — the query recommender, Phase 3 onward — is developed and evaluated against. Accordingly, this report focuses on the data and schema infrastructure the recommender depends on (the two running VKGs, the domain registry, and a schema-format bug fix required to make the Movie domain queryable at all) and omits implementation detail specific to the exploratory GUI used only for local development, which is not part of the system’s architecture going forward.

Contents

1	Introduction	3
1.1	Scope narrowing (post-implementation)	3
2	High-Level System Overview	3
2.1	Overview	3
2.2	In depth: what changed and what did not	4
3	Phase 0 — Data & Infrastructure Preparation	5
3.1	Overview	5
3.2	Movie domain VKG	5
3.3	Tourism domain VKG	6
3.4	Bugs discovered during bring-up	7
3.5	Credential hygiene	8
3.6	Verification	8
4	Phase 1 — Domain-Switching Infrastructure	9
4.1	Overview	9
4.2	The KG registry	9
4.3	The SHACL gap and its fix (Movie domain)	9

4.4	Generated m-schema artifacts	10
4.5	Verification	10
5	Known Limitations and Open Items	12
6	Summary and Next Steps	12

1 Introduction

The base system, Indeewari, allows a non-expert user to type a natural-language question (e.g. “*Churches in Verona*”) and receive results retrieved from a single, pre-wired Virtual Knowledge Graph via an LLM-driven NL2SPARQL pipeline. The thesis extends this system along three axes, in increasing order of research weight:

- (1) **Domain selection** — generalize the application so a user can choose *which* VKG to query (Movie vs. Tourism) instead of the system being wired to a single dataset.
- (2) **User selection instead of login** — let the user pick *who they are* from a list of personas, so the system can reason about that user’s preferences without implementing real authentication.
- (3) **Query recommendation** — the thesis’s core contribution: after answering a query, propose three follow-up query suggestions, conditioned on both the user’s stated queries *and* their real interaction behavior.

This report covers the two phases completed so far: **Phase 0**, which prepares the data and infrastructure needed by every later phase, and **Phase 1**, which delivers item (1) above. Phases 2–6 (user selection, behavioral profiling, the recommender core, evaluation, and thesis writing) are documented separately in the project roadmap and are out of scope here.

1.1 Scope narrowing (post-implementation)

After Phases 0 and 1 were implemented and verified, an email exchange with the supervisor narrowed the thesis scope: items (1) domain selection and (2) user selection instead of login are **not thesis deliverables** — they are owned by a separate application already built by two bachelor students under the same supervisor. The thesis’s entire contribution is item (3), the query recommender, delivered as a **standalone model and API** that any caller (including the bachelor students’ application) can integrate independently. This report is written after that decision and reflects it: the two VKGs and domain-registry infrastructure described below are retained only as an **internal development and evaluation harness** — a way to build and demo the recommender against real schemas and data without depending on the external application’s timeline or codebase. The exploratory Streamlit GUI that was built during Phase 1 to exercise this harness manually is not described here in implementation detail, since it is disposable dev tooling rather than part of the system’s architecture; what matters going forward is that both VKGs are live, queryable, and switchable in configuration, which is what the rest of this report documents.

2 High-Level System Overview

2.1 Overview

Before Phase 0/1, the NL2SPARQL pipeline queried a single, hardcoded SPARQL endpoint with no notion of alternate domains. After these two phases, the system consists of two layers stacked on top of the unchanged pipeline core: a containerized **data/infrastructure layer** (Docker Compose services running Postgres, PostGIS, and two Ontop VKG endpoints), and a **registry layer** (a YAML file listing which knowledge graphs are available, plus a small Python loader). A thin, dev-only harness consumes the registry to point the pipeline’s in-memory configuration (SPARQL endpoint URL, schema path) at whichever domain is being tested; per Section 1.1, this harness is not part of the system’s architecture going forward, so it is omitted from the diagram below. Figure 1 shows how the two retained layers connect to the pipeline.

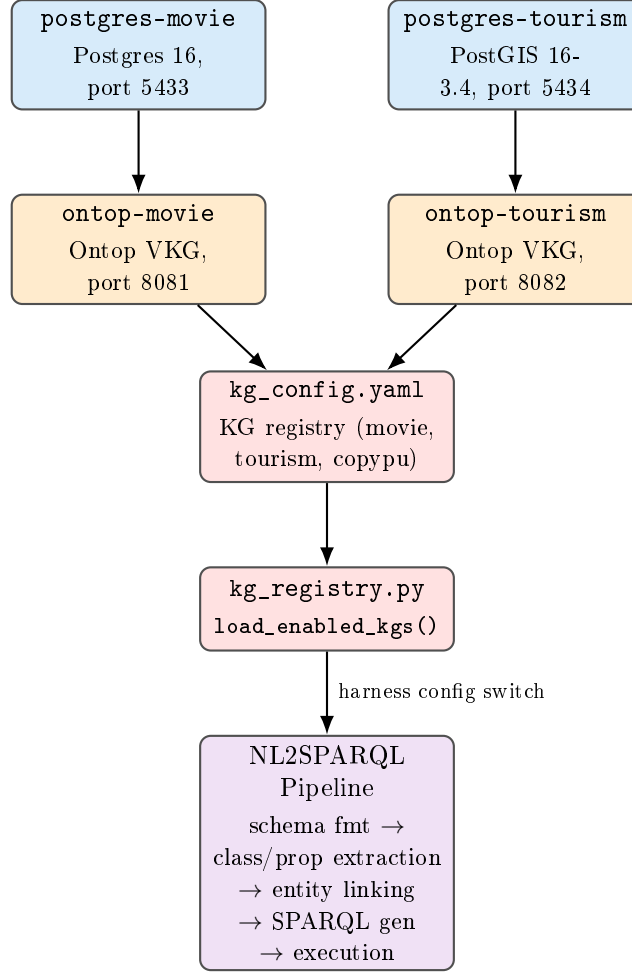


Figure 1: High-level architecture after Phase 0/1: containerized VKGs (bottom-up: database → Ontop endpoint) feed a configuration registry, which a thin dev-only harness (not shown, Section 1.1) reads to reconfigure the unmodified NL2SPARQL pipeline at runtime.

2.2 In depth: what changed and what did not

The NL2SPARQL pipeline itself — schema formatting, class/property relevance extraction, entity linking, SPARQL generation, execution, and result formatting — is **unmodified**. Both phases only add infrastructure and configuration *around* it:

- Phase 0 adds two new SPARQL endpoints the pipeline can point at, each backed by a real relational database exposed as RDF through an Ontop OBDA mapping.
- Phase 1 adds a way to *choose* which endpoint (and which corresponding schema file) the pipeline uses for a given session, replacing what used to be a single hardcoded configuration.

This design choice keeps the two completed phases low-risk: because the pipeline’s internals are untouched, any bug introduced by these phases is necessarily located in the new infrastructure or the new selector code, not in query answering logic that was already tested.

3 Phase 0 — Data & Infrastructure Preparation

3.1 Overview

Phase 0’s goal is to give the system two real, queryable Virtual Knowledge Graphs to switch between in Phase 1: a **Movie** domain (synthetic Netflix-shaped dataset, chosen so it can later carry per-user behavioral signals for the thesis’s recommender) and a **Tourism** domain (a real 293 MB production database dump of Verona’s tourism application, inherited from the Indeewari predecessor project). Each domain is stood up as an isolated Docker Compose stack: a relational database container feeding an Ontop container that exposes the data as a VKG over SPARQL. Figure 2 shows the four services and their dependencies.

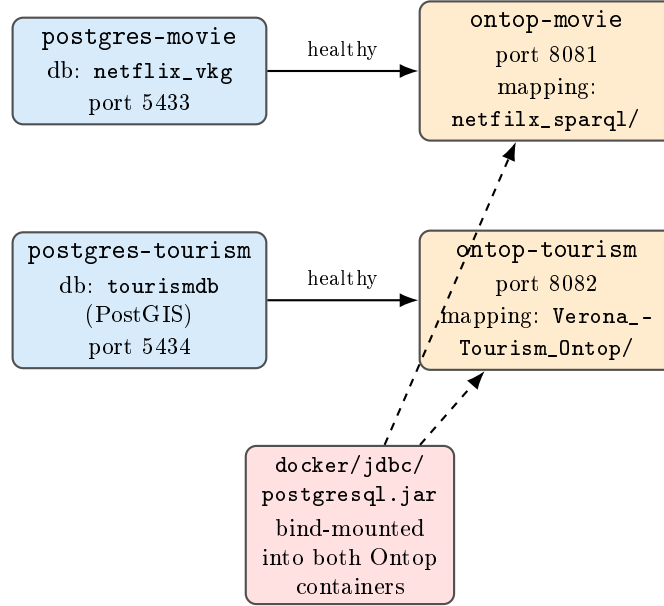


Figure 2: Docker Compose topology for Phase 0. Each Ontop container waits for its database’s healthcheck (`depends_on: service_healthy`) before starting, and both mount the same JDBC driver directory (dashed arrows) to work around a missing-driver issue in the base Ontop image (Section 3.4).

3.2 Movie domain VKG

The Movie side uses the richer of two available Movie datasets in the repository — `netflix_sparql/` (ontology + OBDA mapping) combined with `netflix_dataset/` (the actual CSV data) — rather than a smaller, cleaner but behaviorally-empty alternative (`Movie/`) that has no user, search, or watch-history concept and therefore cannot support the thesis’s later behavioral-profiling work.

`postgres-movie` is initialized from `docker/movie/init.sql`, which creates six tables loaded from the `netflix_dataset/` CSVs:

Table	Rows	Role
movies	1,040	Catalog
users	10,300	User master data
watch_history	105,000	Real behavior (implicit feedback)
search_logs	26,500	Stated intent (free-text search)
reviews	15,450	Explicit stated opinions
recommendation_logs	52,000	System-suggested vs. consumed

Table 1: Movie domain tables in `postgres-movie`, with row counts verified against the source CSVs exactly.

A deliberate design choice: **no table has a primary key**. The source dataset contains genuine duplicate id values across every table (e.g. roughly 40 of the 1,040 `movie_id` values repeat), so plain non-unique indexes are used instead of primary keys, and the init script’s comments record this decision explicitly so it is not mistaken for an oversight later.

Two categories of fixes were required to the pre-existing ontology and OBDA mapping under `netflix_sparql/`:

- **Bug fix:** the `watch-details` mapping in `netflix_ontology.obda` originally selected a column named `watch_duration`, which does not exist — the real column is `watch_duration_minutes`. The mapping was corrected accordingly.
- **Coverage extension:** the mapping set was extended from its original scope to **17 mappings in total**, adding the columns needed by a later phase’s search-to-watch temporal join: `search-details` (`searchDate`), `watch-session-details` (`watchDate`, `action`, `deviceType`), `recommendation-date` (`recommendationDate`), and a new object-property mapping, `watch-session-user`, connecting `WatchSession` to `User` via a new `watchedByUser` property (this relationship did not exist before, despite `WatchSession` → `Movie` and `WatchSession` → `Device` already being present). `netflix_ontology.owl` was updated in lockstep with each new property, annotated with an explanatory comment marking it as an addition.

Listing 1: Excerpt of `docker/movie/init.sql`: the `watch_history` table, whose column naming mismatch with the original OBDA mapping was one of the bugs fixed in this phase.

```
CREATE TABLE watch_history (
  session_id TEXT,
  user_id TEXT,
  movie_id TEXT,
  action TEXT, -- started / paused / stopped / completed
  progress_percentage NUMERIC,
  watch_duration_minutes INTEGER, -- NOTE: not "watch_duration"
  watch_date DATE,
  device_type TEXT,
  user_rating NUMERIC
  -- no PRIMARY KEY: source data has duplicate ids, see script header comment
);
```

3.3 Tourism domain VKG

The Tourism side reuses `Verona_Tourism_Ontop/`, which contains a real production PostgreSQL dump (`tourismdb_jan 1.backup`, 293 MB) of Verona’s tourism application, together with an ontology (`art_all.ttl`) that — unlike either Movie option — already embeds SHACL node shapes per class, a direct structural match for the pipeline’s schema loader (see Section 4.3).

`postgres-tourism` runs on the PostGIS-enabled image (`postgis/postgis:16-3.4`), because the source database uses `geometry/geography` columns that a plain Postgres image cannot represent. It is initialized by `docker/tourism/restore.sh`, which first creates the `postgis` extension and then restores the backup via `pg_restore -no-owner -jobs=2`, tolerating `pg_restore`'s common non-fatal warning exit codes rather than treating them as failures.

Table	Rows
<code>art</code>	41
<code>event</code>	1,416
<code>calendar</code>	10,640
<code>location</code>	1,600

Table 2: Tourism domain tables in `postgres-tourism`, row counts verified against the source backup exactly.

The original OBDA mapping (`art_all.obda`) shipped with 2 broken mappings, `art-image-mapping` and `art-to-image-mapping`, which reference a table `art_images(art_id, image_url, caption, image_type)` that does not exist anywhere in the backup (only a differently-shaped legacy `oldapp.art_media` table exists). Rather than reconstructing the missing table, **these two mappings were dropped entirely** — a decision made explicitly with the professor's/user's sign-off, since art images are not needed for the thesis demo. 14 mappings remain functional:

```
status-mapping, art-mapping, art-category-mapping, art-category-art-mapping,
tour-type-mapping, tour-mapping, tour-art-mapping, event-mapping,
event-category-mapping, event-category-event-mapping, calendar-mapping,
location-mapping, event-to-location-mapping, event-to-calendar-mapping.
```

The classes/properties `ex:ArtImage`, `ex:hasImage`, and `ex:imageOfArt` remain declared in `art_all.ttl` for documentation purposes but are now unpopulated and unused.

Explicit scope decision. The backup also contains a large behavioral table, `log_vc` (4.4 million rows of VeronaCard tap-in visits), which could in principle serve as a behavioral signal analogous to the Movie domain's `watch_history`. However, `log_vc` identifies an anonymous card, not a named person, and the database's only User-like table (`auth_user`) has just 4 admin/staff rows, not tourists. **It was decided with the user that Tourism stays catalog-query-only for this thesis:** `log_vc` is not mapped into the ontology, and all user-selection/behavior-aware work in later phases is carried entirely by the Movie domain. This is recorded here as a load-bearing scope decision, not an oversight, since it bounds what later phases can demonstrate on the Tourism side.

3.4 Bugs discovered during bring-up

Two problems surfaced only once the containers were actually started, and were not anticipated by the original infrastructure design:

1. **Missing JDBC driver.** The base `ontop/ontop` Docker image ships with no JDBC drivers under `/opt/ontop/jdbc/`. Both Ontop containers failed at startup with `Cannot load the driver: org.postgresql.Driver`. **Fix:** the PostgreSQL JDBC driver (version 42.7.4) was downloaded into a new, gitignored directory, `docker/jdbc/`, and bind-mounted into both `ontop-*` services at `/opt/ontop/jdbc` in `docker-compose.yml`.
2. **Comment syntax rejected by the OBDA parser.** Ontop's native `.obda` file parser rejects free-standing `#` comment lines anywhere in the file, not only inside the mapping

block. An explanatory comment describing the two dropped art-image mappings had to be removed from `art_all.obda`; the same explanation is preserved in this report (Section 3.3 above) and in the operational `docker/README.md` instead.

3.5 Credential hygiene

Two legacy Ontop `.properties` files (`netfilx_sparql/netflix_ontology.properties` and `Movie/movie_project.properties`) originally contained plaintext database passwords. Both now carry a `jdbc.password=CHANGEME` placeholder with an explanatory comment — neither file is actually read by the Docker Compose stack, which instead sources real credentials from a gitignored root `.env` file (`POSTGRES_MOVIE_PASSWORD`, `POSTGRES_TOURISM_PASSWORD`), templated in a committed `.env.example`.

3.6 Verification

Phase 0 was verified in two ways:

- **Row-count checks:** every table in Tables 1 and 2 was confirmed to match its source CSV/backup exactly.
- **SPARQL smoke tests:** two manual queries, documented with exact `curl` commands in `docker/README.md` — a movie-title query against `:8081/sparql`, and an `ArtCategory` query against `:8082/sparql` that returns all six category names, including “Chiese” (churches), directly matching the professor’s own example question, “Churches in Verona.”

4 Phase 1 — Domain-Switching Infrastructure

4.1 Overview

Phase 1’s goal is to let the NL2SPARQL pipeline be pointed, at runtime, at either VKG (Movie or Tourism) or the pre-existing `copypu` maritime dataset, replacing what was previously a single hardcoded SPARQL endpoint. As established in Section 1.1, domain selection itself is not a thesis deliverable — it is the responsibility of the bachelor students’ external application — so this section documents only the two pieces of infrastructure that the recommender harness genuinely depends on: a declarative **KG registry** that any caller can read to discover available domains and their endpoints, and a schema-format fix that was required before the Movie domain’s schema could be consumed at all. (A minimal Streamlit control was used locally during development to exercise this registry by hand; its implementation is not documented here, per Section 1.1.) Figure 3 traces how a domain is resolved from the registry to a schema the pipeline can use.

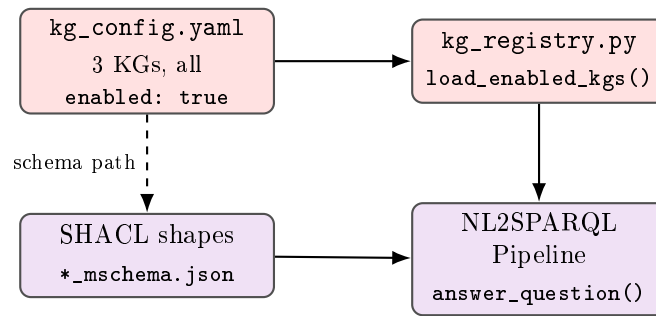


Figure 3: Phase 1 domain resolution: a caller (harness or, eventually, the recommender API) reads the registry to obtain a domain’s endpoint and schema path. The dashed edge denotes that the schema referenced by the registry must already exist as valid SHACL — the gap addressed in Section 4.3 for the Movie domain.

4.2 The KG registry

`web_app/config/kg_config.yaml` lists the available knowledge graphs. Each entry carries a name, a schema file path, a SPARQL endpoint URL, a `source_type`, and an `enabled` flag:

Name	Schema file	Endpoint	source_type
copypu	data/input/copypu.ttl	:8080/sparql	local_rdf
movie	netfilx_sparql/netflix_ontology.owl	:8081/sparql	sparql_endpoint
tourism	Verona_Tourism_Ontop/art_all.ttl	:8082/sparql	sparql_endpoint

Table 3: The three enabled entries in `kg_config.yaml` (a fourth, `dbpedia`, exists commented-out and disabled).

This file previously existed as a documentation-only registry with no code reading it. Phase 1 introduces the first consumer, `web_app/nl2sparql/kg_registry.py`, whose `load_enabled_kgs()` function parses the YAML and returns the entries whose `enabled` flag is true, in file order (an empty list if the file is missing).

4.3 The SHACL gap and its fix (Movie domain)

The pipeline’s only schema loader, `SHACLSchemaParser.parse()` (`nl2sparql/schema_parser.py`), builds the schema entirely from SHACL `sh:NodeShape` triples — it has **no fallback** that reads

plain `rdfs:domain/rdfs:range` declarations. The Movie ontology (`netflix_ontology.owl`) is a plain RDF/XML OWL file with zero SHACL triples, so feeding it directly to the parser silently produced an **empty schema** — the Movie domain would have appeared selectable in the GUI but answered every question incorrectly.

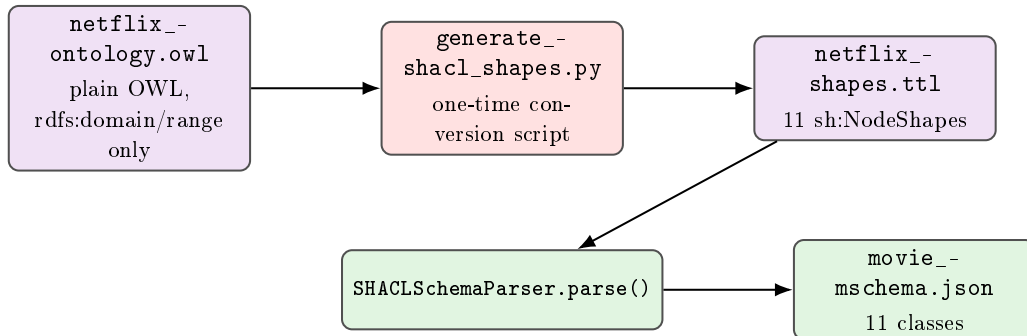


Figure 4: The Movie ontology has no SHACL shapes, so a one-time script generates them from its existing `rdfs:domain/range` declarations before the pipeline’s schema parser can consume it. The Tourism ontology needed no such step, since `art_all.ttl` already embeds SHACL shapes directly.

The fix is a new one-time script, `netflix_sparql/generate_shacl_shapes.py`: for every `owl:Class` it emits an `sh:NodeShape` and `sh:targetClass`; for every property whose `rdfs:domain` matches that class, it emits an `sh:property` blank node, using `sh:class + sh:nodeKind sh:IRI` for object properties, or `sh:datatype` for datatype properties. The output, `netflix_sparql/netflix_shapes.ttl`, contains 11 node shapes — one per class (Country, Device, Genre, Language, Movie, Recommendation, Review, Search, SubscriptionPlan, User, WatchSession).

4.4 Generated m-schema artifacts

Both domains’ machine-readable schema representations (the “m-schema” JSON format the pipeline’s later stages consume) were generated via `pipeline.extract_schema()` and committed under `web_app/data/output/`:

- `movie_mschema.json` — 11 classes, matching the 11 generated node shapes. Four classes (Country, Device, Genre, Language, SubscriptionPlan) carry only a class label with no properties, since nothing else in the ontology targets them as an object-property range.
- `tourism_mschema.json` — 7 classes (e.g. `example:Art`, `example:ArtCategory`), generated directly from the already-SHACL-shaped `art_all.ttl` with no intermediate conversion step required.

4.5 Verification

Phase 1 was verified along two lines, both without requiring an LLM API key (no key was configured in the verification environment, so only the non-LLM stages of the pipeline were exercised directly):

1. **Endpoint correctness:** direct SPARQL queries against both live Ontop endpoints confirmed the generated shapes’ property IRIs match real data — e.g. `ex:title` on `ex:Movie` returns real titles, and the Tourism `ArtCategory` query still returns “Chiese.”
2. **Schema-stage correctness:** running the pipeline’s embedding-based class/property extraction stage against both new schemas correctly identified `Movie/Genre + hasGenre/`

`title/releaseYear` for “*What movies have the genre Action?*”, and `Art/ArtCategory/Event + artname_it`/etc. for “*Churches in Verona.*”

Verification of the dev-only Streamlit control used to exercise this registry by hand (domain-switch logic, browser reachability) is not reported here, consistent with Section 1.1 excluding that tooling from the system’s documented architecture.

5 Known Limitations and Open Items

- **Tourism has no per-user behavioral signal**, by explicit design decision rather than oversight (see the Tourism scope decision, Section 3.3): this bounds what later phases (user selection, behavior-aware recommendation) can demonstrate on the Tourism side — that work is carried entirely by the Movie domain.
- **Neither Phase 0 nor Phase 1 is a thesis deliverable** as of the scope decision in Section 1.1. Both are retained solely as the data/infrastructure harness that Phase 3 onward (the recommender) is built and evaluated against.
- `kg_config.yaml` is currently consumed only by the dev-only harness described in Section 1.1; no other part of `web_app`, and no part of the recommender API itself, reads it yet. The eventual API contract for domain selection (if any) is an open question tracked in the project roadmap, not resolved by this report.

6 Summary and Next Steps

Phases 0 and 1 deliver a working, verified foundation: two real VKGs running behind Ontop, switchable via a declarative registry, without touching the NL2SPARQL pipeline’s core logic. Per the scope decision in Section 1.1, this foundation is now purely a development harness; the thesis’s actual contribution builds on top of it:

- **Phase 3** (complete, documented separately in *Phase 3 Implementation Report*) — a behavioral preference profile extending the Movie domain’s `watch_history/search_logs` data into a “stated vs. actual preference” signal, the empirical basis for the thesis’s core contribution.
- **Phase 4** — the query recommender itself: LLM-based candidate generation grounded in both query history and the Phase 3 behavioral signal, with schema-validation and ranking before three suggestions are shown to the user, delivered as a standalone API.
- **Phases 5–6** — evaluation and thesis writing.

Full detail on these phases is tracked in the project roadmap document, *Thesis Project Plan*, which also documents the four literature references (ADBIS 2026, GQR/RA-GQR, the LLM+KG roadmap survey, and QRMOCCGA) that ground the recommender’s design.